Mini Paper — 2.3 Algorithms — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Q1 [2] (AO1).

- Big O describes how an algorithm's **time (or space) requirement grows as the input size n increases**, for the worst case (1).
- Constants/lower-order terms are ignored because we care about the **rate of growth / behaviour for large n**, where the dominant term decides scalability (1).

Examiner tip: always tie the definition to "as n increases" — a bare "how fast an algorithm is" scores nothing.

Q2 [3] (AO1).

- (a) **O(n)** (1).
- (b) **O(log n)** (1).
- (c) **O(2^n)** (1).

Examiner tip: "doubles with each extra item" is the textbook signal for exponential O(2^n) — do not write O(n^2).

Q3 [5] (AO2). Insertion sort on `[6, 2, 5, 3]`:

i	key	Before	Action	After
1	2	`[6,2,5,3]`	shift 6 right, insert 2	`[2,6,5,3]`
2	5	`[2,6,5,3]`	shift 6 right, insert 5	`[2,5,6,3]`
3	3	`[2,5,6,3]`	shift 6,5 right, insert 3	`[2,3,5,6]` ✓

- Correct result after inserting 2 → `[2,6,5,3]` (1).
- Correct result after inserting 5 → `[2,5,6,3]` (1).
- Correct result after inserting 3 → `[2,3,5,6]` (1).
- Shows shifting of larger elements rather than just swapping / clear working (1).
- Worst-case time complexity **O(n^2)** (1).

Examiner tip: insertion sort builds the sorted part at the front — show the list after each insertion, not just the final sorted list.

Q4 [4] (AO2). Binary search for 19 in `[3,7,11,15,19,23,27]`:

Step	low	high	mid	list[mid]	Comparison	Action
1	0	6	3	15	15 < 19	low = 4
2	4	6	5	23	23 > 19	high = 4
3	4	4	4	19	19 == 19	**found at index 4**

- Correct mid sequence **3, 5, 4** (1).
- Correct low/high updates (low=4 then high=4) (1).
- Found at **index 4** after **3 comparisons** (1).
- Precondition: the list must be **sorted** (1).

Examiner tip: mid = (low + high) DIV 2 — integer division. Show every low/high update; marks are for the working, not just "found".

Q5 [4] (AO1/AO2).

- (a) **O(n²)** (1).
- (b) The outer loop runs n times and for **each** outer iteration the inner loop runs up to n times, giving up to **n × n = n²** operations; the n² term dominates so the complexity is O(n²) (2).
- (c) **Bubble sort** OR **insertion sort** (either accepted) (1).

Examiner tip: justify O(n²) with the multiplication "n passes × n comparisons" — a nested loop is the classic source of quadratic time.

Q6 [4] (AO2). Traversals of the BST:

- Pre-order (Node, Left, Right): **45, 25, 15, 35, 60, 55, 70** (1).
- In-order (Left, Node, Right): **15, 25, 35, 45, 55, 60, 70** (1).
- Post-order (Left, Right, Node): **15, 35, 25, 55, 70, 60, 45** (1).
- The in-order output is in **ascending sorted order** (1).

Examiner tip: remember the rule by where the Node sits — Pre = node first, In = node middle (sorted), Post = node last. Deduct for any out-of-place node.

Q7 [8] (AO3). Dijkstra's algorithm, A → E. Tentative distances from A ("—" = settled/visited):

Step	Current	A	B	C	D	E	Visited
Init	–	0	∞	∞	∞	∞	{}
1	**A** (0)	—	4	∞	**1**	∞	{A}
2	**D** (1)	—	min(4, 1+2)=**3**	∞	—	min(∞, 1+7)=8	{A,D}
3	**B** (3)	—	—	min(∞, 3+3)=**6**	—	min(8, 3+6)=**9**	{A,D,B}
4	**C** (6)	—	—	—	—	min(9, 6+2)=**8**	{A,D,B,C}
5	**E** (8)	—	—	—	—	—	{A,D,B,C,E}

- Correct initialisation: A = 0, all others = ∞ (1).
- A settled first; D relaxed to 1, B relaxed to 4 (1).
- Visits nodes in correct distance order **A, D, B, C, E** (2).

- Correct relaxation of B to 3 via D, and C to 6 via B (2).
- Shortest distance **A → E = 8** (1).
- Shortest path **A → D → B → C → E** (1 + 2 + 3 + 2 = 8), found by tracing predecessors E ← C ← B ← D ← A (1).

Examiner tip: always pick the smallest unvisited tentative distance next, and never re-open a settled node. State both the distance (8) AND the path — many candidates drop the easy path mark.

Total: 30 marks.